

Git Interview Preparation Notes

SDE & Full Stack Developer Prep

Contents

1	Phase 1: The Core Architecture & Local Basics	1
1.1	Theory: What is Git?	1
1.2	Theory: The “Three Trees” Architecture	1
1.3	Command Snippets: Foundational Commands	2
2	Phase 2: Parallel Development (Branching & Merging)	2
2.1	Theory: Branches and Merge Conflicts	2
2.2	Command Snippets: Branching & Merging	2
3	Phase 3: Remote Collaboration	3
3.1	Theory: Fetch vs. Pull	3
3.2	Command Snippets: Remotes	3
4	Phase 4: Advanced Workflows (Time Travel & Undoing)	3
4.1	Theory: Reset vs. Revert & Rebase vs. Merge	3
4.2	Command Snippets: Advanced Commands	4

1 Phase 1: The Core Architecture & Local Basics

1.1 Theory: What is Git?

Git is a Distributed Version Control System (DVCS). It tracks changes to files over time and ensures that every developer has a full, independent copy of the project history on their local machine.

1.2 Theory: The “Three Trees” Architecture

Git manages files using three distinct areas:

- **Working Directory:** Your actual file system where you write code.
- **Staging Area (The Index):** A “loading dock” where you group related changes before permanently saving them.
- **Local Repository (.git folder):** Git’s database where committed snapshots are permanently stored.

1.3 Command Snippets: Foundational Commands

```
# Initialize a new Git repository
git init

# Check the status of the Three Trees
git status

# Move a specific file to the Staging Area
git add index.html

# Move all modified/new files to the Staging Area
git add .

# Permanently save staged files to the Local Repository
git commit -m "Initial commit"

# View commit history
git log
```

2 Phase 2: Parallel Development (Branching & Merging)

2.1 Theory: Branches and Merge Conflicts

A branch in Git is not a duplicate copy of files; it is simply a lightweight, movable pointer to a specific commit. This allows for isolated feature development. A merge conflict occurs when Git cannot automatically resolve differences during a merge (e.g., two branches modify the exact same line). You must manually edit the file to resolve the conflict, stage the file, and commit.

2.2 Command Snippets: Branching & Merging

```
# List all local branches
git branch

# Create a new branch (without switching)
git branch feature-header

# Switch to an existing branch
git switch feature-header
# (or use the older command: git checkout feature-header)

# Create and switch to a new branch in one command
git switch -c feature-header

# Merge a feature branch into the main branch
git switch main
```

```
git merge feature-header
```

3 Phase 3: Remote Collaboration

3.1 Theory: Fetch vs. Pull

This is a critical architectural distinction:

- **git fetch:** Safely downloads new commits from the remote repository to your local machine *without* altering your Working Directory.
- **git pull:** An aggressive command that executes a **fetch** and then immediately forces a **merge** into your active Working Directory, which can trigger unexpected conflicts.

3.2 Command Snippets: Remotes

```
# Download a remote repository to your local machine
git clone <repository_url>

# Link a local repository to a new remote server
git remote add origin <repository_url>

# Push local commits to the remote branch
# (-u establishes the tracking connection for the first time)
git push -u origin main

# Safely download remote updates without merging
git fetch

# Download and immediately merge remote updates
git pull
```

4 Phase 4: Advanced Workflows (Time Travel & Undoing)

4.1 Theory: Reset vs. Revert & Rebase vs. Merge

- **Reset vs. Revert:** Use **reset** to undo local, unpushed mistakes (it rewrites history). Use **revert** to undo public, pushed mistakes (it creates a new commit that cancels out the error, preserving history).
- **Rebase vs. Merge:** **Merge** ties branches together chronologically with a merge commit. **Rebase** rewrites history to place your feature branch at the very tip of the main branch, creating a clean linear history. **Never rebase a public branch.**

4.2 Command Snippets: Advanced Commands

```
# -- STASHING (Shelving messy work) --
git stash          # Save uncommitted changes
git stash pop      # Reapply shelved changes
git stash list     # View all stashes

# -- UNDOING MISTAKES --
# Undo commit but keep files staged
git reset --soft HEAD~1

# Undo commit and un-stage files (Default)
git reset --mixed HEAD~1

# WARNING: Erase commit and delete all local changes
git reset --hard HEAD~1

# Safely undo a public commit by creating a reverse commit
git revert <commit-hash>

# -- ADVANCED INTEGRATION --
# Rebase your current branch onto main
git rebase main

# Grab one specific commit and apply it to the current branch
git cherry-pick <commit-hash>
```